Abstract

In this study, a Hidden Markov Model (HMM) was developed to identify the Kunitz-type protease inhibitor domain, which is found in proteins that regulate the activity of proteolytic enzymes. Initially, a set of 158 proteins containing the Kunitz domain was extracted from the PDB database using the PFAM identifier PF00014, corresponding to the Kunitz/Bovine pancreatic trypsin inhibitor domain. To ensure data quality, clustering and filtering steps were performed to select non-redundant representative sequences. Additionally, Structural Multiple Alignment analysis was conducted using the online tool `pdbefold` to identify key structural patterns in the sequences. The output of all these processes included 24 representative sequences, which were used to train the HMM model.

After building the model using HMMER hmmbuild and extracting test data from SwissProt, the HMMER hmmsearch step was applied to the test datasets to identify significant matches between the trained HMM profile and test protein sequences from positive and negative datasets. The output files (`.out`) were then processed using command-line tools to generate `.class` files containing `E-value` scores and other key metrics used to evaluate model accuracy. Subsequently, the `performance.py` script was executed to calculate metrics such as `Q2`, `TPR`, `PPV`, and `MCC`, providing insight into the model's performance.

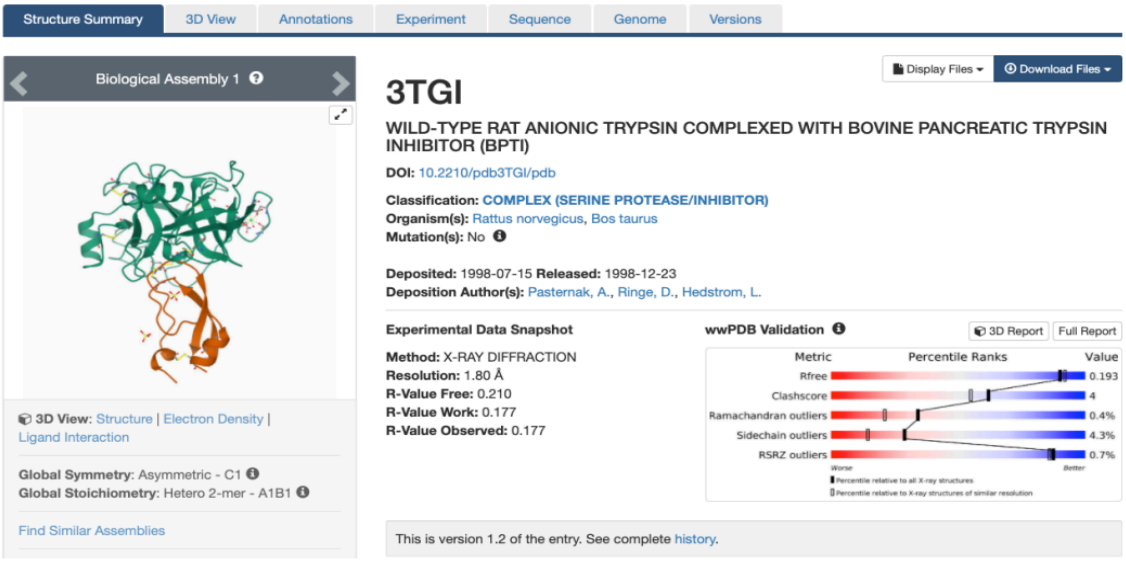
The results indicated that selecting an E-value threshold of `1e-6` achieved an optimal balance between correctly identifying positive sequences and minimizing model errors. This study highlights the significant role of E-value threshold adjustment in improving HMM classification accuracy, demonstrating that proper threshold selection can directly impact model performance. These findings could contribute to optimizing bioinformatics models and medical applications related to protease inhibitor identification.

Introduction

Kunitz domains are active regions in proteins that act as protease inhibitors, meaning they block and regulate the function of enzymes that break down proteins. Controlling protease activity is important for maintaining biological balance. If these enzymes become too active, they can damage healthy tissues, cause chronic inflammation, and contribute to diseases like cancer and Alzheimer's. Protease inhibitors, such as Kunitz domains, help regulate this process and prevent excessive protease activity from causing harm.

Examples of these inhibitors include Aprotinin (BPTI), Amyloid Precursor Protein (APP), and Tissue Factor Pathway Inhibitor (TFPI). Aprotinin is a protease inhibitor that specifically uses the Kunitz domain to block enzymes like trypsin, chymotrypsin, and plasmin. This domain has a stable structure that allows it to bind to these enzymes and stop their activity. Because of these features, aprotinin is used in medicine, especially in heart surgeries, to reduce bleeding.

Aprotinin works by using its inhibitory loop to bind to the active site of trypsin. The lysine-15 residue of aprotinin fits into the specific pocket of trypsin's active site, effectively blocking the enzyme's function and preventing the breakdown of important proteins.



The 3TGI complex, visible on the PDB website, provides a highly detailed structural model of the binding between wild-type rat anionic trypsin and bovine pancreatic trypsin inhibitor (BPTI). This structure precisely shows the mechanism of enzymatic inhibition.

Kunitz inhibitors play an important role in biological processes, but finding these domains in new protein sequences can be difficult. Standard sequence searches often cannot recognize small differences in domain structure, which can result in mistakes—some real Kunitz domains might be ignored(false negatives). In contrast, others that aren't Kunitz domains might be wrongly identified (false positives). Hidden Markov Models (HMMs) help solve this problem by detecting hidden patterns in sequences, allowing them to recognize even slight changes in domain structure more accurately. They use a mathematical approach to find conserved patterns that define Kunitz domains.

This study focuses on using HMMs to identify Kunitz-type protease inhibitors, combining both sequence and structural information. By creating an HMM profile, training it with key sequences, and adjusting classification settings like E-value thresholds, we aim to make domain detection more precise. The results of this research can improve the identification of protease inhibitors in bioinformatics databases and support medical applications like drug development and disease studies.

Materials and Methods

1) Preparing Data for Training the Model

a) Collecting training data: Using the PDB website to find and extract proteins that have the Kunitz domain (PF00014 from Pfam). Selection is based on sequence length (40 to 80 amino acids) and structure resolution (less than 3.5 angstroms).

b) Reducing duplicate sequences using CD-HIT to group similar sequences and remove repeated or highly similar ones.

c) Filtering unique sequences (those that have no similar matches) from the CD-HIT output. Finally, 24 representative sequences with the highest similarity to other members of their clusters were selected. Very long sequences, like 2ODY_E, were manually removed.

d) Multiple Structural Alignment: Using PDBeFold to compare 3D structures of representative proteins and analyze spatial similarities, even if their sequences are different.

e) Multiple Sequence Alignment (MSA) was also performed using MUSCLE on the EMBL-EBI website to compare and arrange amino acid sequences accurately and identify conserved regions.

2) **Building a Hidden Markov Model (HMM):** Using hmmbuild to create a statistical model based on representative proteins that were structurally aligned using PDBeFold, to identify hidden patterns in sequence structures.
*Hmmbuild **pdb_kunitz_nr_clean.hmm** pdb_kunitz_nr_clean.ali*

3) **Preparing Data for Testing the Model**

a) Collecting data: Gathering Kunitz-positive proteins (`xref:pfam-pf00014` and `reviewed: true`) from the UniProt website, including 398 proteins.

b) Downloading all reviewed proteins from the Swiss-Prot database on UniProt.

c) Removing positive proteins from the full dataset to create a negative protein set.

d) Removing proteins with high similarity (95%) and overlap (50 amino acids) to the training dataset from the test set.

i) Created a database with all 398 positive proteins from UniProt.

ii) Compared them with 24 representative (***pdb_kunitz_nr.fasta***) sequences using this database

iii) Used Linux commands to find IDs of highly similar proteins, identifying 32 proteins.

iv) These IDs were used in a Python script (***get_seq.py***) to generate the final positive protein file, excluding these 32 proteins.

e) Creating two positive sets and two negative sets, ensuring equal numbers from the positive and negative protein datasets.

f) Searching with HMM model: Applying `hmmsearch` to identify significant matches between the trained HMM profile and test protein sequences from positive and negative datasets. For instance:
hmmsearch -Z 1000 --max --tblout pos_1.out ../HMM/pdb_kunitz_nr_clean.hmm pos_1.fasta
(Output of this command is pos_1.out)
grep -v "^#" pos_1.out |awk '{split($1, a, "\|") ; print a[2],1,$5,$8}' |tr " " "\t" > pos_1.class

(Output of this command is pos_1.class)
And do it for other positive and negative data sets

g) Merging each positive set with a negative set to create two final datasets in `.class` format, which are suitable for predictive modeling and machine learning analysis. (***set_1.class*** and ***set_2.class***)

4) **Model Testing and Evaluation**

a) After creating two sets of combinations of positive and negative data, model Performance Evaluation is performed: Using a Python script (***performance.py***) to calculate and check the model performance's metrics (these metrics are discussed in the "Results and discussion" section)

5) **All files and scripts** for this project can be accessed from this **link**.

Results and discussion

In the final model testing stage, a for loop was used to run the ***performance.py*** script with different E-values on the two datasets (***set_1.class*** and ***set_2.class***). The for loop ran 12 times, testing E-values from 0.1, 0.01, down to 1e-12. This is testing different E-value values to find the best balance between detecting positive cases and avoiding errors. Overall, the model's performance is acceptable. The result of running the Python script is shown in the image below.

*for i in 'seq 1 12'; do python **performance.py** set_1.class 1e-$i; done*

TH	Q2	MCC	TPR	PPV	CM
$1e^{-1}$	**0.9998813677599442**	**0.9186601672859828**	**0.8440366972477065**	**1.0**	**[[286382, 0], [34, 184]]**
$1e^{-2}$	**0.999940683879972**	**0.9567489489700443**	**0.9154228855721394**	**1.0**	**[[286399, 0], [17, 184]]**
$1e^{-3}$	**0.9999860432658758**	**0.9892975971352793**	**0.9787234042553191**	**1.0**	**[[286412, 0], [4, 184]]**
$1e^{-4}$	**0.9999930216329379**	**0.9946056525388577**	**0.989247311827957**	**1.0**	**[[286414, 0], [2, 184]]**
$1e^{-5}$	**0.9999965108164689**	**0.9972918941022746**	**0.9945945945945946**	**1.0**	**[[286415, 0], [1, 184]]**
$1e^{-6}$	**1.0**	**1.0**	**1.0**	**1.0**	**[[286416, 0], [0, 184]]**
$1e^{-7}$	**0.9999965108164689**	**0.9972771655630667**	**1.0**	**0.9945652173913043**	**[[286416, 1], [0, 183]]**
$1e^{-8}$	**0.9999965108164689**	**0.9972771655630667**	**1.0**	**0.9945652173913043**	**[[286416, 1], [0, 183]]**
$1e^{-9}$	**0.9999965108164689**	**0.9972771655630667**	**1.0**	**0.9945652173913043**	**[[286416, 1], [0, 183]]**
$1e^{-10}$	**0.9999965108164689**	**0.9972771655630667**	**1.0**	**0.9945652173913043**	**[[286416, 1], [0, 183]]**
$1e^{-11}$	**0.9999965108164689**	**0.9972771655630667**	**1.0**	**0.9945652173913043**	**[[286416, 1], [0, 183]]**
$1e^{-12}$	**0.9999895324494068**	**0.9918091292092845**	**1.0**	**0.9836956521739131**	**[[286416, 3], [0, 181]]**

*for i in 'seq 1 12'; do python **performance.py** set_2.class 1e-$i; done*

TH	Q2	MCC	TPR	PPV	CM
$1e^{-1}$	**0.9998639218422889**	**0.9082949717777876**	**0.8251121076233184**	**1.0**	**[[286377, 0], [39, 184]]**
$1e^{-2}$	**0.9999616189811584**	**0.9713668227750496**	**0.9435897435897436**	**1.0**	**[[286405, 0], [11, 184]]**
$1e^{-3}$	**0.9999825540823447**	**0.9864447630067864**	**0.9837837837837838**	**0.9891304347826086**	**[[286413, 2], [3, 182]]**
$1e^{-4}$	**0.9999755757152826**	**0.9809178567341356**	**0.9836065573770492**	**0.9782608695652174**	**[[286413, 4], [3, 180]]**
$1e^{-5}$	**0.9999860432658758**	**0.9890638036334939**	**1.0**	**0.9782608695652174**	**[[286416, 4], [0, 180]]**
$1e^{-6}$	**0.9999860432658758**	**0.9890638036334939**	**1.0**	**0.9782608695652174**	**[[286416, 4], [0, 180]]**
$1e^{-7}$	**0.9999825540823447**	**0.9863108559202272**	**1.0**	**0.9728260869565217**	**[[286416, 5], [0, 179]]**
$1e^{-8}$	**0.9999825540823447**	**0.9863108559202272**	**1.0**	**0.9728260869565217**	**[[286416, 5], [0, 179]]**
$1e^{-9}$	**0.9999825540823447**	**0.9863108559202272**	**1.0**	**0.9728260869565217**	**[[286416, 5], [0, 179]]**
$1e^{-10}$	**0.9999685973482205**	**0.9752215500189169**	**1.0**	**0.9510869565217391**	**[[286416, 9], [0, 175]]**
$1e^{-11}$	**0.9999685973482205**	**0.9752215500189169**	**1.0**	**0.9510869565217391**	**[[286416, 9], [0, 175]]**
$1e^{-12}$	**0.9999616189811584**	**0.9696294565487017**	**1.0**	**0.9402173913043478**	**[[286416, 11], [0, 173]]**

Model performance evaluation criteria

1. **Accuracy (Q2 Score)**: Measures how correct the model's predictions are overall.
    a. float(CM[0][0]+CM[1][1])/(sum(CM[0])+sum(CM[1]))
    b. In both datasets, as the E-value decreases, the model's accuracy increases up to the threshold of 1e-6, then starts to decline. Therefore, at E-value = 1e-6, the model achieves its highest accuracy.
    c. In both datasets, accuracy increases up to the threshold of 1e-6, because lower E-values usually help to find patterns with a higher accuracy.
    d. Additionally, the drop in accuracy after this threshold is also expected because the model may become too sensitive and classify invalid signals as positives
    e. In set_2.class, there is one exception to this trend: when transitioning from E-value 0.001 to 0.0001, instead of an increase, accuracy drops. This could have reasons as below, which may all lead to lower accuracy at certain thresholds
        i. Less Protein Diversity
        ii. Sequence Overlap Pattern: Sequences in this dataset may be too similar
        iii. Effect of Removed Data: Filtering out highly similar proteins may have altered the statistical features of the dataset
        iv. Conservative Model Behavior: At certain E-value thresholds, the model may reject true positives
2. **Matthews Correlation Coefficient (MCC):** A number that shows how well the model makes correct predictions, considering both true and false positives and negatives.
    a. d=(CM[0][0]+CM[1][0])*(CM[0][0]+CM[0][1])*(CM[1][1]+m[1][0])*(CM[1][1]+CM[0][1])
    MCC = (CM[0][0]*CM[1][1]-CM[0][1]*CM[1][0])/math.sqrt(d)

    b. Before the 1e-6 threshold: As E-value decreases, the model improves accuracy and class separation *(without being overly strict)*, leading to higher MCC.
    c. After the 1e-6 threshold: Once the E-value reaches 1e-6, the model likely becomes too sensitive, leading to misclassifications.
3. **TPR: True Positive Rate (CM[1,1]):** Measures how well the model correctly identifies positive cases.
4. **Positive Predictive Value (PPV)**: Measures how many of the predicted positive cases are correct.
5. **Confusion Matrix (CM = [[TNR, FNR], [FPR, TPR]]):** A table that shows how well the model predicts positive and negative cases, including correct and incorrect predictions. (The sum of false predictions, which is CM[0,1] + CM[1,0], is decreasing until the threshold E-value: 1e-6, and at this point, the sum of false values is minimum(highest accuracy), and after the threshold, this is again increasing, which shows a reduction in accuracy. We are again witnessing the previously mentioned exception in passing from 0.001 to 0.0001 in dataset set_2.class)
    a. **TN: True Negative(CM[0, 0]):** shows how many negatives have been identified correctly by the model
    b. **FN: False Negative(CM[0, 1]):** shows how many positives have been identified wrongly as negatives
    c. **FP: False Positive (CM[1,0]):** shows how many negatives have been identified wrongly as positives
    d. **TP: True Positive (CM[1,1])**: shows how many positives have been identified correctly by the model

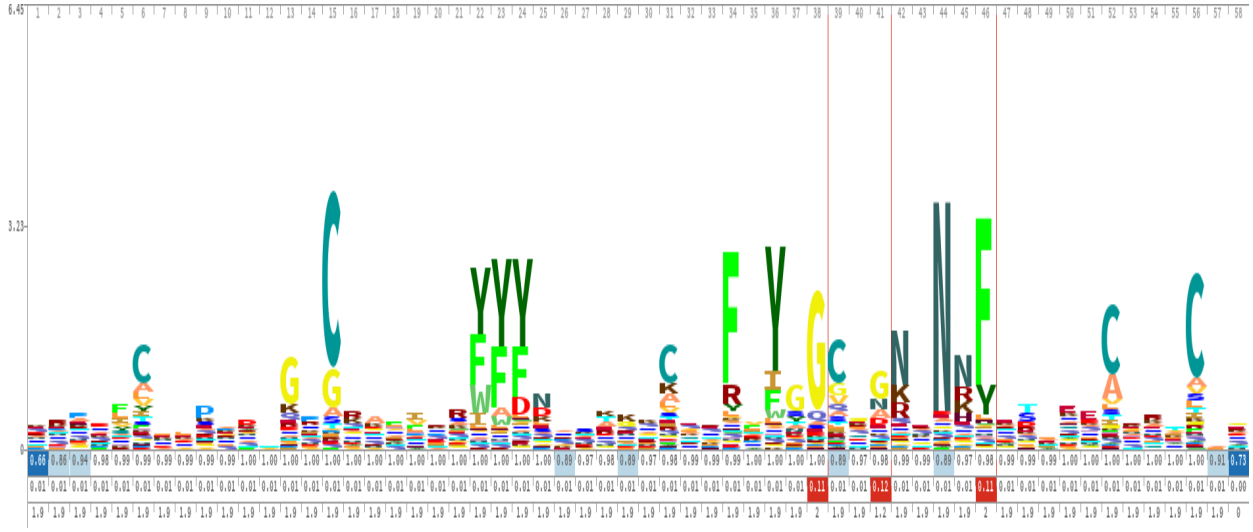
Cross-validation

A method used to evaluate the performance of machine learning models by assessing their ability to generalize to unseen data and avoid overfitting to a specific dataset. In this project, to assess the robustness of the HMM-based Kunitz domain detector, two independent test sets were used. The first set was employed to determine an appropriate E-value threshold for domain detection, while the second set was used to verify whether the model maintained reliable performance under the same threshold. This approach ensures that the model does not merely perform well on a single dataset but can generalize its predictions across different data sources. Relying on only one test set could lead to biased evaluation, where the model is overly optimized for that specific data and fails to perform adequately on new inputs—a common symptom of overfitting. Therefore, this two-step evaluation aligns conceptually with the goals of cross-validation and strengthens the credibility of the model's results.

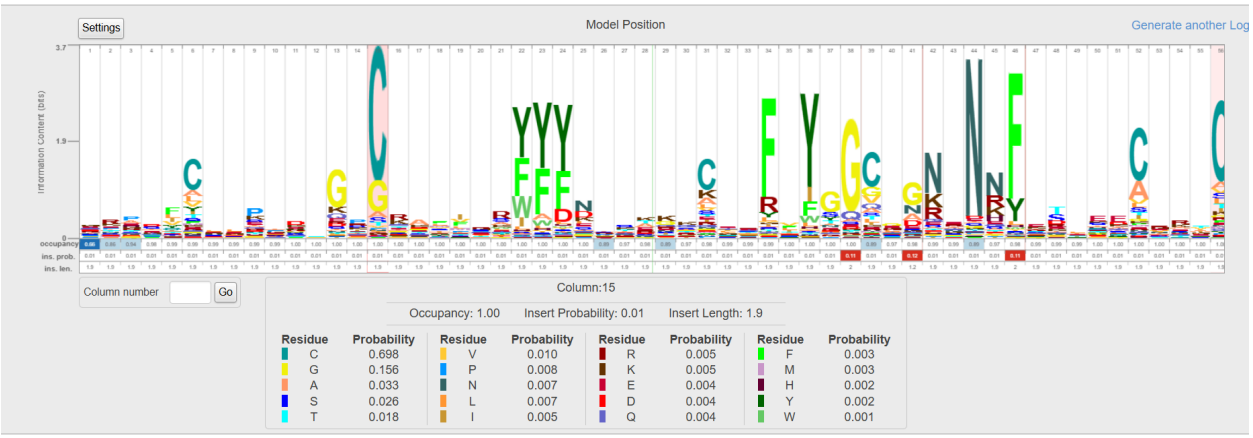
HMM Logo
Metadata:

Alphabet	**AA**
Letter Height	**Information Content - All**
Number of Sequences	**24**
Length	**58**
Uploaded File Name:	**pdb_kunitz_nr_clean.hmm**
Created	**Sat Jun 7 12:25:41 2025**

This **number 58** defines the analysis range, meaning the model has finally used these aligned 58 positions (as more important and informative sites) for training and learning



As an example, detailed information on column 15 (**C or Cysteine in position 15**)



The relation between the HMM logo & HMM model

The HMM logo is a picture that shows what the model learned from the training data. It is based on the aligned protein sequences used to build the model. The logo helps us see which positions in the protein sequences gave the most useful information. The height of each letter shows how often that amino acid appears in that spot. Bigger letters mean more important. So, the logo gives us a clear and easy way to understand what the model focuses on when learning to find the Kunitz domain. For instance, the amino acid **C (Cysteine) in position 15** appears very tall and clear. This means it was present in many training sequences at that spot, so the model learned that it is important. It shows that this position helps the model recognize Kunitz domains more accurately.

Model improvement

- **Weighting more important spots in the HMM model:** We can give more weight to the important positions in the sequence, which we see clearly in the HMM logo. These are positions where the same amino acid appears in many training sequences, and they help the model find the Kunitz domain better. For example, after using `hmmsearch`, we can check if the test

sequence has the important amino acids in those key positions. If it doesn't, we can reduce its score or ignore it. This can help reduce wrong results and make the model more accurate.

- **Applying structural filtering before collecting test data:** In this study, the **" test data"** were selected based only on sequence similarity. To improve model accuracy, it is suggested to also use 3D structural data or predicted structures (such as from AlphaFold or DSSP) when choosing **"negative samples"**. This can help reduce false positives and lead to more precise model performance. However, care should be taken not to over-filter the test data, as removing too many difficult cases may lead to biased results. The goal is to reduce noise without limiting the model's ability to generalize.

Conclusion

In this study, we used a Hidden Markov Model (HMM) to find proteins that have the Kunitz domain. The steps of the project were explained earlier, and here we only give a summary of the results. The final model showed good ability to tell apart positive and negative proteins. This means the model was trained well, and the training data gave enough useful information. In the test step, we used a Python script to check the model with different E-values. The results showed very high accuracy: 1.0 for set_1.class and 0.99998 for set_2.class.

We also looked at the HMM logo to find which parts of the sequence had more useful information. Then, we gave two suggestions to make the model better: (1) Give more importance to the strong positions in the sequence when building the model again, and (2) Use 3D structure filters when choosing *only* the negative test data, similar to how we used structure alignment (with PDBeFold) in the training step.

The first suggestion helps the model focus on important positions and improves its accuracy. The second one helps remove noise from the test data so we can measure the model more clearly. Instead of choosing negative test proteins only by sequence, we also look at their structure to avoid mistakes. But when we give suggestions like these, we should always keep a balance — we want the test data to stay realistic and fair, without making it too easy or too cleaned up.

References

https://www.uniprot.org/

https://www.rcsb.org/

https://skylign.org/

https://www.ebi.ac.uk

https://colab.research.google.com/drive/1QvcAh8Ha6iVCZGIfpom_JVhkpsPFAxyg

By Sareh Salehi, Student Number: 0001164160
MSc student in Bioinformatics
University of Bologna

June 2025